SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA3703 – Software Testing

ASSIGNMENT 5 – CO5

Apply, Analyze and Justify Software Testing Principles and Defect Management Strategies for a University Student Portal

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Student Name	K. ANJALI GIRI	**Register Number**	192421037
Department / Section	B.TECH- IT	**Academic Year / Semester**	3RD YEAR
Faculty Name	DR.R. GNANAJEYARAMAN	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	4	4	Fault detection 4.0% / 80.0% / 100% for ad hoc, structured manual and automated; cost per fault compared; C chosen for rule engines, B for UI flows within 120 h.

Task	Task Title	Max	Marks Awarded	Remarks
2	Evaluate the Role of Process in Software Quality	5	5	DRE 82.9% / 92.0% / 94.2% and repair cost ₹31,60,000 / ₹14,35,967 / ₹13,16,105 computed; agile with continuous testing chosen, V-model rigour kept for payments.
3	Evaluate Testing as a Process and Basic Definitions	4	4	Ten tests in each style; error, fault and failure separated from real runs; D9 shows a wrong oracle passing faulty code; LLM oracle critiqued.
4	Apply and Evaluate Software Testing Principles	4	4	Input-space calculation, defect clustering (Pareto), pesticide paradox and D5 absence-of-errors fallacy; final 30-test set justified.
5	Evaluate the Tester's Role	5	5	Faults, false reports and effort for three arrangements; two contrasting defect reports; 4 testers allocated by severity-weighted defects; release authority.
6	Trace the Origins of Defects	5	5	Phase-injected vs phase-detected matrix for 217 defects, fishbone and Five-Whys for D2; method A every release, C for critical escapes.
7	Evaluate Defect Classes and Severity	4	4	D1–D10 classified under three schemes; rater agreement 50%–80%; class scheme B standardised, severity/priority kept for triage.
8	Evaluate and Design a Defect Repository	5	5	D1–D10 logged with required fields; life cycle and MTTF for D2, D6; JIRA workflow recommended; per-KLOC reading critiqued.

Task	Task Title	Max	Marks Awarded	Remarks
9	Integrate Defect Examples into a Testing Practice	6	6	Four configurations compared: detection 0.6% / 66.7% / 100% / 83.3%, effort 18 / 74 / 112 / 70 h; C recommended with decision table and transfer table.
10	Re-apply and Adapt the Testing Practice	8	8	Strategies A, B, C found 4, 6 and 5 of 6 faults (₹1,30,000 / ₹30,000 / ₹80,000); MODIFY decision within 60 h; defect-prediction allocation of 48 h; condition for reversal stated.
TOTAL		**50**	**50**	**Percentage: 100%**

OVERALL REMARKS

Model answer. Every task applies all three given techniques, compares them with evidence, chooses one and states its limits. Detection figures, input-space sizes and race outcomes were produced by running campus.py and exp1–exp3.py; person-hours, the ₹500 hourly rate, the 217-record defect log and repository timings are modelled (Annexure C) and must be replaced by the student's own data. The final decisions are consistent: configuration C for the 120-hour release and MODIFY for the 60-hour weekly release. Screenshots of Bugzilla / JIRA in Task 8 must be taken from the real tools.

K. ANJALI GIRI		
Student Signature	**Faculty Signature**	**Program Director**

TASK 1 – MEASUREMENTS AND MILESTONES: COMPARE MEASUREMENT FRAMEWORKS

1.1 Aim

To analyze and compare different software testing measurement frameworks and identify suitable metrics for monitoring the quality and progress of the SureClaim Motor Insurance Claims Platform v2.0.

1.2 Introduction

Software testing measurements help an organization understand whether the testing process is progressing according to the project plan. Measurements provide numerical information about test execution, defects, test coverage, quality, and resource usage. Without proper measurements, the release manager may not know whether the software is ready for release or whether corrective action is required.

SureClaim v2.0 introduces several important features, including mobile claim intake, photo upload, automated damage estimation, fraud detection, claim settlement payout, and a garage-partner portal. Since these features handle customer information and financial transactions, the testing process must be carefully monitored.

1.3 Objectives

1. To understand different software testing measurement frameworks.
2. To calculate testing progress for weeks 5 and 9.
3. To calculate pass rate, defect find rate, closure rate, and defect density.
4. To define measurable release milestones.
5. To identify early warning signs from weekly testing data.
6. To compare the advantages and disadvantages of measurement frameworks.
7. To prepare a Goal–Question–Metric tree for release quality.

1.4 Overview of the Given Project Data

The SureClaim v2.0 release contains approximately 22 KLOC of new or modified code and 96 requirements. A total of 1,400 test cases are planned over a 12-week release period. Testing is

scheduled from week 1 to week 9, while release-readiness activities are planned for weeks 10 to 12.

The project team contains 15 members, including a release manager, testers, developers, a configuration engineer, a business analyst, and a QA analyst.

The weekly cumulative testing data is shown below.

Week	Planned tests	Executed tests	Passed tests	Defects found	Defects closed
1	120	100	88	15	5
2	300	240	200	42	20
3	520	410	340	80	46
4	740	560	450	130	78
5	960	720	590	178	118
6	1150	900	750	214	156
7	1280	1080	920	238	190
8	1360	1230	1075	252	224
9	1400	1340	1210	260	240

1.5 Comparison of Measurement Frameworks

A. Plan-versus-Actual Measurements

Plan-versus-actual measurement compares the planned testing activities with the activities completed. It helps the release manager understand whether the team is following the expected schedule.

Important measurements include:

- Planned test cases.
- Executed test cases.
- Percentage of planned tests completed.
- Difference between planned and executed tests.

- Effort spent compared with planned effort.
- Completion of important project milestones.

For example, if 960 tests are planned by week 5 but only 720 tests are executed, the testing team is behind schedule.

Advantages:

1. Easy to understand and calculate.
2. Clearly shows the difference between planned and completed work.
3. Helps identify schedule delays.
4. Supports resource planning.
5. Makes milestone monitoring easier.

Limitations:

1. Completing a large number of test cases does not guarantee good software quality.
2. Test cases may be executed without sufficient coverage.
3. The measurement does not directly show the seriousness of open defects.
4. A team may focus on completing test cases instead of testing important risks.

B. Defect-Based Measurements

Defect-based measurements focus on defects discovered, fixed, and remaining in the application. They help the team understand the quality of the software.

Important measurements include:

- Number of defects found.
- Number of defects closed.
- Number of open defects.
- Defect arrival rate.
- Defect closure rate.
- Defect density.

- Defect removal efficiency.

For example, SureClaim has 260 cumulative defects found and 240 defects closed at week 9. Therefore, 20 defects remain open.

Advantages:

1. Provides information about software quality.
2. Helps identify modules with frequent defects.
3. Supports defect prioritization.
4. Helps monitor the team's defect-fixing progress.
5. Provides information for release decisions.

Limitations:

1. A high number of defects does not always mean poor testing.
2. A low number of defects may mean that testing is incomplete.
3. Defect severity must be considered along with defect count.
4. Testers may be encouraged to report more defects instead of finding important defects.

C. Coverage and Risk Measurements

Coverage measurements show how much of the software has been tested. Risk measurements identify areas that may cause serious problems if they are not tested properly.

Important measurements include:

- Requirement coverage.
- Statement coverage.
- Branch coverage.
- Risk coverage.
- Number of untested requirements.
- Number of high-risk requirements without completed tests.

At week 9, SureClaim has 90% requirement coverage, 78% statement coverage, and 64% branch coverage.

Advantages:

1. Shows whether requirements are tested.
2. Identifies untested areas.
3. Helps focus testing on high-risk features.
4. Provides information about code execution.
5. Reduces the possibility of missing important requirements.

Limitations:

1. High coverage does not guarantee that the software is defect-free.
2. Coverage does not always measure the quality of test cases.
3. Code coverage tools may not identify all business logic problems.
4. Risk ratings may depend on human judgment.

1.6 Comparison of Week 5 and Week 9 Results

Measurement	**Week 5**	**Week 9**
Planned tests	960	1400
Executed tests	720	1340
Execution percentage	75%	95.71%
Pass rate	81.94%	90.30%
Schedule variance	-240	-60
Defects found	178	260
Defects closed	118	240
Defect find rate	24.72%	19.40%
Defect closure rate	66.29%	92.31%

Open defects	60	20
Defect density	8.09	11.82

The comparison shows that testing progress improved between week 5 and week 9. The execution percentage increased from 75% to 95.71%, and the defect closure rate increased from 66.29% to 92.31%.

However, the defect density increased from 8.09 to 11.82 defects per KLOC. The project target is a maximum of 10 defects per KLOC. Therefore, the target was exceeded by week 9.

The release manager should not consider the application ready only because most test cases have been executed. The remaining critical defects and coverage gaps must also be considered.

1.9 Release Milestones

Milestones are important checkpoints used to monitor whether project activities are completed within the planned time.

Milestone	**Target completion condition**	**Actual result**	**Status**
M1 – Initial test readiness	100 tests executed by week 1	100 executed	Met
M2 – Early testing progress	At least 75% of planned tests executed by week 5	75% executed	Met
M3 – Major test execution	At least 90% of planned tests executed by week 7	84.38% executed	Not met
M4 – Test completion	At least 95% of planned tests executed by week 9	95.71% executed	Met
M5 – Release quality readiness	No open critical defects and all high-risk requirements tested	3 critical defects remain open; only 90% requirement coverage	Not met

The milestones show that the team achieved the test execution target by week 9. However, the quality-readiness milestone was not achieved because three critical defects remained open and ten requirements had not been covered by executed tests.

1.10 Early Warning Analysis

The weekly data provides several warning signs.

The first warning appears in week 1. Only 100 out of 120 planned tests were executed.

[frac{100}{120}\times100=83.33%]

This means that the team completed only 83.33% of its planned work in the first week.

The schedule gap continued to increase:

Week	Planned	Executed	Gap
1	120	100	20
2	300	240	60
3	520	410	110
4	740	560	180
5	960	720	240
6	1150	900	250
7	1280	1080	200
8	1360	1230	130
9	1400	1340	60

The schedule gap reached its highest value of 250 tests in week 6. It later decreased as testing progress improved.

Defect density became another warning sign. The density exceeded the target of 10 defects per KLOC in week 7:

[frac{238}{22}=10.82]

Therefore, the schedule warning was visible from week 1, while the defect-density target was exceeded in week 7. The early schedule measurement provided a warning six weeks before the week 7 defect-density problem became visible.

1.11 Risks of Misusing Measurements

Plan-versus-actual measurement: If testers are rewarded only for completing test cases, they may execute simple test cases first and ignore difficult, high-risk scenarios. This creates an incorrect impression of progress.

Defect-based measurement: If testers are rewarded for reporting more defects, they may report many minor issues instead of concentrating on serious defects. Defect severity and impact must therefore be considered.

Coverage and risk measurement: If the team focuses only on achieving 100% code coverage, it may execute code without properly checking the expected results. Coverage should not be treated as proof that the software is correct.

1.12 Goal–Question–Metric Tree

Goal: Release SureClaim v2.0 with no open critical defects.

Goal	Question	Metric
Ensure critical defects are resolved	How many critical defects remain open?	Open critical defect count
Improve defect fixing	How quickly are defects closed?	Defect closure rate
Verify all important requirements	How many high-risk requirements have executed tests?	High-risk requirement coverage
Confirm test completion	What percentage of planned tests are executed?	Test execution percentage
Verify test quality	What percentage of executed tests pass?	Pass rate
Monitor software quality	How many defects exist per KLOC?	Defect density

Automatic Collection of Metrics

Metric	Possible tool	Automatic collection
Test execution percentage	Test-management tool	Yes

Pass rate	Test-management tool	Yes
Open defect count	JIRA/Bugzilla	Yes
Defect closure rate	JIRA/Bugzilla	Yes
Defect density	JIRA + code-size data	Partly
Requirement coverage	Test-management tool	Partly
Open critical defects	JIRA/Bugzilla	Yes

Metrics such as test execution, pass rate, and open defect count can be collected automatically when the tools are configured properly. Requirement coverage and defect density may require linking multiple tools or maintaining additional information.

1.13 Comparison and Final Measurement Selection

The plan-versus-actual approach is useful for schedule monitoring, defect-based measurement is useful for quality monitoring, and coverage measurement is useful for identifying untested areas.

For SureClaim, a combination is more useful than depending on only one approach.

The final measurement set contains eight metrics:

1. Test execution percentage.
2. Test pass rate.
3. Schedule gap.
4. Open critical defect count.
5. Total open defect count.
6. Defect closure rate.
7. Requirement coverage.
8. Defect density.

These metrics provide a balanced view of schedule, quality, and test completeness. They can be collected using the available test-management tool, JIRA/Bugzilla, Git, and a simple reporting script.

1.14 Result

The measurement frameworks were compared and the testing metrics for weeks 5 and 9 were calculated. The results show that test execution improved to 95.71% by week 9, but the project still had 20 open defects, including three critical defects. Defect density also exceeded the specified target.

1.15 Conclusion

This task demonstrated the importance of selecting appropriate measurements for software testing. Plan-versus-actual measurements help monitor schedule progress, defect-based measurements help understand software quality, and coverage measurements help identify untested requirements.

For SureClaim v2.0, the combination of all three approaches provides better control than using a single measurement framework. The final eight-metric set helps the release manager identify problems early and make informed release decisions.

TASK 2 – STATUS REPORTS: COMPARE STATUS REPORTING FORMATS

2.1 Aim

To prepare and compare different software testing status reporting formats and identify the most suitable reporting method for monitoring SureClaim v2.0.

2.2 Introduction

A software testing status report communicates the progress, quality, risks, and problems of a testing project. It allows project managers and testing teams to understand what has been completed and what activities still require attention.

In a large project, sending long weekly emails may not be effective because important information can be missed. A clear and structured report helps management identify problems quickly and take corrective action.

This task compares three reporting formats: Red–Amber–Green summary, metric-chart pack, and a daily tool-generated dashboard.

2.3 Objectives

1. To understand different testing status reporting formats.
2. To prepare three status reports for week 6.
3. To define colour rules for reporting.
4. To apply the rules to weeks 3, 6, and 9.
5. To compare the information provided by each format.
6. To identify the most useful reporting format for management and the testing team.

2.4 Comparison of Reporting Formats

A. Red–Amber–Green Summary

The Red–Amber–Green method uses three colours to communicate project status.

- Green indicates that the activity is progressing as planned.
- Amber indicates that there is a warning or possible delay.
- Red indicates that immediate attention is required.

The report usually includes schedule, scope, quality, and risk.

Advantages:

1. Easy for management to understand.
2. Highlights important problems.
3. Supports quick decision-making.

Limitations:

1. Does not provide detailed numerical information.
2. Colour selection may depend on personal judgment.
3. A green status may hide problems that are not included in the report.

B. Metric-Chart Pack

A metric-chart pack presents testing information through charts and tables. It can include planned versus executed tests, passed tests, defect arrivals, defect closures, and open defects by severity.

Advantages:

1. Shows trends over time.
2. Helps identify schedule delays.
3. Makes comparisons easier.
4. Provides numerical evidence.

Limitations:

1. Requires time to prepare.
2. Large numbers of charts may confuse readers.
3. Management may need assistance to understand technical measurements.

C. Daily Tool-Generated Status

A tool-generated dashboard displays live testing information from tools such as JIRA, Bugzilla, and test-management applications.

It can show:

- Open defects.
- Closed defects.
- Test execution status.
- Failed test cases.
- Defect severity.
- Assigned team members.
- Pending activities.

Advantages:

1. Provides frequently updated information.
2. Reduces manual reporting.
3. Helps testers monitor daily activities.
4. Provides direct access to detailed records.

Limitations:

1. Requires correct tool configuration.
2. Incorrect data entry may affect dashboard accuracy.
3. Management may not regularly check the dashboard.
4. A dashboard alone may not explain the reason behind a problem.

2.5 Week 6 Status Report – Format A: RAG Summary

SureClaim v2.0 – Week 6 Status

Reporting period: End of week 6.

Area	**Measurement**	**Status**	**Comment**
Schedule	900 of 1150 tests executed	Red	250 tests behind plan

Test quality	750 of 900 tests passed	Amber	Pass rate is 83.33%
Defects	58 open defects	Red	Defect backlog remains high
Coverage	Coverage data available at week 9 only	Amber	Week 6 coverage requires separate collection
Release risk	Testing is behind plan	Red	Corrective action is required

Colour Rules

The following rules are used:

- **Green:** Less than or equal to 5% of planned tests are not executed, no open critical defects, and quality targets are achieved.
- **Amber:** More than 5% but not more than 15% of planned tests are not executed, or a quality measurement needs attention.
- **Red:** More than 15% of planned tests are not executed, or any critical defect remains open beyond the permitted period.

The assignment does not provide the age of critical defects. Therefore, the critical-defect age rule cannot be confirmed from the weekly data. The schedule rule can still be applied.

RAG Status for Weeks 3, 6 and 9

Week	Planned	Executed	Unexecuted	Unexecuted percentage	Schedule status
3	520	410	110	21.15%	Red
6	1150	900	250	21.74%	Red
9	1400	1340	60	4.29%	Green

Although week 9 has a green schedule status, the release cannot receive an overall green quality status because three critical defects remain open.

2.6 Week 6 Status Report – Format B: Metric-Chart Pack

The metric-chart pack contains the following charts.

Chart 1: Planned Tests versus Executed Tests

Week	Planned tests	Executed tests
1	120	100
2	300	240
3	520	410
4	740	560
5	960	720
6	1150	900
7	1280	1080
8	1360	1230
9	1400	1340

The chart shows that the number of executed tests remained below the planned number throughout the testing period. The gap increased until week 6 and later started decreasing.

Chart 2: Passed Tests

Week	Passed tests
1	88
2	200
3	340
4	450
5	590
6	750
7	920
8	1075
9	1210

The number of passed tests increased continuously. At week 6, 750 tests had passed out of 900 executed tests.

[\text{Pass Rate}=\frac{750}{900}\times100]

[=\boxed{83.33%}]

Chart 3: Defects Found versus Defects Closed

Week	Defects found	Defects closed
1	15	5
2	42	20
3	80	46
4	130	78
5	178	118
6	214	156
7	238	190
8	252	224
9	260	240

At week 6, 214 defects had been found and 156 had been closed.

Chart 4: Open Defects

Week	Open defects
1	10
2	22
3	34
4	52
5	60
6	58

7	48
8	28
9	20

The defect backlog increased up to week 5 and then gradually decreased. This indicates that defect closure activities became more effective after week 5.

Week 6 Metric Summary

Metric	**Value**
Planned tests	1150
Executed tests	900
Execution percentage	78.26%
Passed tests	750
Pass rate	83.33%
Defects found	214
Defects closed	156
Open defects	58
Schedule gap	250

2.7 Week 6 Status Report – Format C: Daily Dashboard

A daily dashboard can be designed in JIRA or a test-management tool.

The dashboard should contain the following sections:

Test Execution Panel

- Total planned tests: 1,400.
- Executed tests by week 6: 900.
- Passed tests: 750.
- Remaining tests against the week 6 plan: 250.

Defect Panel

- Total defects found: 214.
- Total defects closed: 156.
- Open defects: 58.

Progress Panel

- Execution percentage: 78.26%.
- Pass rate: 83.33%.
- Schedule status: Red.

Action Panel

1. Identify the reasons for the 250-test schedule gap.
2. Review defects that remain open.
3. Prioritize critical and major defects.
4. Check whether unstable builds or environment issues are delaying testing.
5. Update the dashboard after every test execution and defect status change.

A short daily meeting can be held to discuss blocked tests, important defects, and activities planned for the next working day.

2.8 What Each Format Reveals and Hides

Reporting format	**Reveals**	**Hides**
RAG summary	Overall health and urgent problems	Detailed trends and calculations
Metric-chart pack	Progress trends, defect movement and schedule gaps	Reasons behind each change unless explained
Daily dashboard	Current testing activity and individual defect records	Management-level explanation and overall project context

The metric-chart pack would show the widening schedule gap most clearly because it compares planned and executed tests over several weeks. The dashboard can also show the gap, but only if the required historical charts are configured.

2.9 Effort and Audience Comparison

Format	Preparation effort	Main audience	Main risk
RAG summary	Low	Release manager and senior management	Oversimplification
Metric-chart pack	Medium	Management, QA and testing leads	Too many measurements
Daily dashboard	Initial setup is medium; daily maintenance is low if automated	Testers, developers and QA	Incorrect or incomplete tool data

2.10 Critique of an LLM-Generated Status Report

An AI-generated status report can help prepare a first draft, but every statement must be checked against the actual data.

For example, the following statements require correction or evidence:

Example statement	Evaluation
“All testing is completed by week 9.”	Incorrect. Only 1,340 of 1,400 planned tests were executed.
“The pass rate is 95% at week 9.”	Incorrect. The actual pass rate is approximately 90.30%.
“There are no open defects.”	Incorrect. There are 20 open defects.
“The schedule is fully recovered.”	Partly incorrect. The schedule gap has reduced to 60 tests, but the planned total is not yet completed.
“The release is ready for production.”	Unsupported. Three critical defects remain open.

“All requirements have been tested.”	Incorrect. Requirement coverage is 90%, or 86 out of 96 requirements.

AI-generated reports must be treated as drafts. Numerical calculations, defect severity, coverage, and release recommendations should be verified by the QA analyst.

2.11 Final Reporting Recommendation

The release manager should receive a one-page RAG summary supported by a small metric-chart pack every week. This gives management a clear overview while preserving numerical evidence.

The testing team should use a daily tool-generated dashboard. This allows testers and developers to track execution progress, defects, and blocked activities without preparing long emails.

2.12 Result

Three reporting formats were prepared and compared. The week 6 report showed that 78.26% of the planned tests had been executed and 58 defects remained open. The RAG rules were applied to weeks 3, 6, and 9.

2.13 Conclusion

Status reporting is an important part of software testing because it communicates progress and quality to the people responsible for the release. A RAG report is suitable for management, a metric-chart pack is useful for trend analysis, and a live dashboard is useful for daily team monitoring.

For SureClaim, a combination of a weekly RAG summary, supporting charts, and a daily dashboard provides a clear and practical reporting system.

TASK 3 – TEST MONITORING AND CONTROL: COMPARE CONTROL RESPONSES TO A SLIPPING TEST SCHEDULE

3.1 Aim

To analyze the delay in the SureClaim testing schedule and compare different control responses that can be used to improve testing progress.

3.2 Introduction

Test monitoring and control involve checking the actual testing progress against the project plan and taking corrective action when delays or quality problems occur.

During week 5, SureClaim executed only 720 tests against 960 planned tests. This represents a shortfall of 240 tests. The gap continued to increase and reached 250 tests in week 6.

The release manager must decide how to control the delay without reducing the quality of the application.

Three possible approaches are considered:

1. Add additional testing effort.
2. Re-prioritize testing and defer low-risk test cases.
3. Identify and remove the actual causes of testing delays.

3.3 Objectives

1. To calculate the weekly schedule gap.
2. To calculate the weekly test execution rate.
3. To estimate the completion time using the week 5 execution rate.
4. To simulate three control responses.
5. To compare cost, quality risk, and team impact.
6. To recommend a suitable control action.

3.4 Weekly Gap Calculation

[\text{Testing Gap}=\text{Planned Tests}-\text{Executed Tests}]

Week	Planned	Executed	Gap
1	120	100	20
2	300	240	60
3	520	410	110
4	740	560	180
5	960	720	240
6	1150	900	250
7	1280	1080	200
8	1360	1230	130
9	1400	1340	60

The largest gap occurred in week 6, with 250 tests behind plan. After week 6, the gap decreased, showing that execution progress improved.

3.5 Weekly Execution Rate

Weekly execution rate is calculated by finding the increase in cumulative executed tests between two weeks.

Week	Cumulative executed	Tests executed during week
1	100	100
2	240	140
3	410	170
4	560	150
5	720	160
6	900	180
7	1080	180
8	1230	150

9	1340	110

Total tests executed during the nine weeks:

[1340]

Average weekly execution rate:

[=\frac{1340}{9}]

[=\boxed{148.89\text{ tests per week}}]

The execution rate during week 5 was:

[720-560=\boxed{160\text{ tests}}]

At the week 5 rate, 680 tests remained after week 5.

[1400-720=680]

[\text{Additional Weeks Required}=\frac{680}{160}]

[=\boxed{4.25\text{ weeks}}]

Therefore, if the week 5 execution rate continued, the planned 1,400 tests would be completed approximately 4.25 weeks after week 5, or during week 10.

3.6 Control Response A – Add Testing Effort

Under this approach, the project adds two temporary testers for four weeks and provides overtime to the existing team.

The assignment suggests that two additional testers may increase productivity by 60% after one week of training. For the simulation, the following assumptions are used:

- Existing execution rate: 160 tests per week.
- Week 6 is treated as the training period.
- From week 7, productivity increases by 60%.
- The maximum planned test count is 1,400.

Week	Expected execution	Cumulative execution
5	Already completed	720

6	160	880
7	256	1136
8	256	1392
9	8	1400

Under these assumptions, the testing team can complete the planned test count by week 9.

Advantages:

1. Increases testing capacity.
2. Helps reduce schedule delays.
3. Provides additional support during high workload.
4. May help complete remaining tests earlier.

Disadvantages:

1. Additional testers increase project cost.
2. Temporary testers require training.
3. New team members may initially produce inconsistent results.
4. Overtime may reduce team morale and increase fatigue.

Adding people is useful when insufficient testing capacity is the main reason for delay. It is less effective when the actual problem is unstable builds or blocked environments.

3.7 Control Response C – Remove the Cause of Delay

This approach focuses on identifying the reasons for testing delays.

Possible causes include:

- Test environment downtime.
- Unstable software builds.
- Slow defect fixing.
- Missing or incorrect test data.
- Delays in requirement clarification.

- Repeated test execution due to software changes.
- Communication problems between developers and testers.

The team should record blocked test cases and classify their reasons. The release manager can then arrange corrective actions.

Possible cause	**Corrective action**
Environment downtime	Improve environment monitoring and maintenance
Unstable builds	Introduce build verification tests
Slow defect fixing	Prioritize critical and major defects
Missing test data	Prepare and validate test data earlier
Unclear requirements	Arrange clarification meetings with the business analyst
Repeated test failures	Perform root-cause analysis
Delayed communication	Use daily defect and testing meetings

For the simulation, assume that removing the main blockers increases the weekly execution rate by 20% from week 6.

[160\times1.20=192]

The expected progress is:

Week	**Expected execution**	**Cumulative execution**
5	Already completed	720
6	192	912
7	192	1104
8	192	1296
9	104	1400

The team can complete the planned 1,400 tests by week 9 under this simplified simulation.

This result is based on the assumption that the corrective actions are effective and the remaining tests are available for execution.

3.9 Comparison of the Three Control Responses

Factor	Add effort	Re-prioritize	Remove cause
Cost	High	Low to medium	Medium
Schedule improvement	High	High for reduced scope	High if blockers are removed
Quality risk	Depends on training and test quality	Higher if important tests are deferred	Lower when root causes are corrected
Team morale	Overtime may reduce morale	Can reduce workload	May improve team confidence
Root-cause correction	Not guaranteed	Not guaranteed	Main objective
Long-term benefit	Limited	Limited	High

3.10 Defect Backlog Analysis

Open defects are calculated as:

[\text{Open Defects}=\text{Defects Found}-\text{Defects Closed}]

Week	Found	Closed	Open defects
1	15	5	10
2	42	20	22
3	80	46	34
4	130	78	52
5	178	118	60
6	214	156	58
7	238	190	48
8	252	224	28

9	260	240	20

The defect backlog increased from 10 in week 1 to 60 in week 5. This suggests that defects were being discovered faster than they were being closed.

After week 5, the number of open defects decreased. This indicates that defect fixing and closure activities improved.

However, the weekly cumulative data does not identify the exact control actions that caused the improvement. Therefore, the relationship between the improvement and any particular action cannot be confirmed without additional project records.

3.11 Simulation of Defect and Closure Effects

For a simple comparison, assume that:

- The average defect discovery ratio from week 5 is 48 new defects per 160 tests.
- The defect-fixing team can close approximately 38 defects per week after week 5.
- New defects and closures depend on actual test execution and fixing capacity.
- These figures are illustrative planning assumptions, not measured future results.

The estimated defect arrival ratio is:

[\frac{48}{160}=0.30]

If 680 remaining tests are executed:

[680\times0.30=204]

Approximately 204 additional defects may be discovered under this simplified assumption.

The expected open backlog is:

[60+204-152=112]

Here, 152 represents 38 closures per week over four weeks.

This estimate demonstrates that completing more test cases does not automatically eliminate the defect backlog. The defect-fixing capacity must also be managed.

3.12 Evaluation and Final Control Decision

The recommended approach is to combine Response C with limited use of Response A.

The project should first identify and remove the main causes of delay. If testing capacity is still insufficient after blockers are corrected, temporary testers can be added for selected high-risk activities.

Response B should be used only for carefully approved low-risk test cases. Critical fraud, payout, security, and claim settlement tests should remain within the release scope.

3.13 Judging the Statement: "Adding People to a Late Test Effort Makes It Later"

Adding people to a delayed project can increase communication and training effort. New members may require time to understand the application and testing procedures.

However, the statement is not always true. If the work can be divided properly and the main problem is a lack of testing capacity, additional testers can improve progress.

In this simulation, additional testers increase productivity after a training period and help complete the planned test count. Therefore, adding people can help when the conditions are suitable.

3.14 Result

The weekly schedule gap, execution rate, and projected completion time were calculated. Three control responses were simulated and compared. The analysis showed that removing the actual causes of delay, supported by limited additional resources, is the most suitable approach.

3.15 Conclusion

Test monitoring and control help identify delays and take corrective action before they affect the release. The SureClaim data showed a major schedule gap by week 5 and a peak gap of 250 tests in week 6.

The comparison demonstrated that adding resources, reducing low-risk scope, and removing blockers can all improve progress under different conditions. A combined approach that first removes the causes of delay and then adds limited support is recommended for SureClaim.

TASK 4 – CRITERIA FOR TEST COMPLETION: COMPARE STOPPING CRITERIA

4.1 Aim

To compare different software testing completion criteria and establish a suitable stopping rule for the SureClaim v2.0 release.

4.2 Introduction

Software testing cannot continue indefinitely because every project has limitations in time, cost, and resources. However, stopping testing too early may allow serious defects to reach production.

A clear stopping criterion helps the release manager decide whether sufficient testing has been completed. It should consider test execution, pass rate, defect trends, open defect severity, test coverage, and the remaining risk.

This task compares three approaches: test-case completion, defect-trend stopping, and risk-and-cost-based stopping.

4.3 Objectives

1. To understand different testing stopping criteria.
2. To apply each criterion to the SureClaim data.
3. To analyze the weekly defect discovery trend.
4. To estimate the remaining undetected defects.
5. To compare the cost of releasing immediately with the cost of further testing.
6. To recommend a suitable stopping rule.

4.4 Comparison of Stopping Criteria

A. Test-Case Completion

This method stops testing when:

- 100% of planned tests have been executed.
- At least 95% of executed tests have passed.

Advantages:

1. Easy to understand.
2. Provides a clear completion target.
3. Supports test planning.
4. Helps prevent incomplete execution.

Limitations:

1. Does not directly consider defect severity.
2. Does not guarantee that all important risks are covered.
3. A large number of passed tests does not guarantee that the application is safe to release.

B. Defect-Trend Stopping

This method stops testing when the weekly defect discovery rate falls below a selected threshold for two consecutive weeks and no critical defect remains open.

For this task, the threshold is assumed to be fewer than 10 newly discovered defects per week.

Advantages:

1. Helps monitor defect discovery trends.
2. Provides evidence about the stability of the software.
3. Considers whether the number of new defects is decreasing.

Limitations:

1. Defect discovery may fall because testing activity has reduced.
2. It may be affected by incomplete test coverage.
3. It must be combined with defect severity and coverage information.

C. Risk-and-Cost-Based Stopping

This method compares the expected cost of defects remaining after release with the cost of continuing testing.

Testing should continue when the expected benefit of finding additional defects is greater than the cost of further testing.

Advantages:

1. Considers business risk.
2. Helps management make informed decisions.
3. Supports prioritization of critical functions.
4. Considers the financial impact of defects.

Limitations:

1. Requires assumptions about undetected defects.
2. The financial impact of defects may be difficult to estimate.
3. Cost calculations cannot replace mandatory safety or regulatory requirements.

4.5 Applying Test-Case Completion to Week 9

The planned test count is 1,400, while 1,340 tests have been executed.

[\text{Execution Percentage}=\frac{1340}{1400}\times100]

[=\boxed{95.71%}]

The pass rate is:

[\text{Pass Rate}=\frac{1210}{1340}\times100]

[=\boxed{90.30%}]

The execution target is not met because 100% of planned tests have not been executed.

The pass-rate target is also not met because the pass rate is below 95%.

Therefore, the test-case completion criterion does not permit testing to stop at week 9.

4.6 Applying Defect-Trend Stopping

The weekly new defects are calculated by finding the difference between consecutive cumulative defect counts.

Week	Cumulative defects	New defects during week
1	15	15

2	42	27
3	80	38
4	130	50
5	178	48
6	214	36
7	238	24
8	252	14
9	260	8

The defect discovery rate fell below 10 during week 9 only. It did not remain below 10 for two consecutive weeks.

In addition, three critical defects remain open.

Therefore, the defect-trend criterion does not permit testing to stop at week 9.

4.7 Estimating Remaining Defects

A simple declining trend can be used to prepare an approximate forecast.

The new defects in the final two weeks were:

- Week 8: 14.
- Week 9: 8.

The approximate weekly ratio is:

$$r=\frac{8}{14}$$

$$r=0.5714$$

If the same declining ratio continued, the future defect discoveries may be estimated as:

$$8(0.5714)+8(0.5714)^2+8(0.5714)^3+\cdots$$

Using a geometric series:

$$\text{Estimated Future Defects} =\frac{8r}{1-r}$$

[=\frac{8(0.5714)}{1-0.5714}]

[\approx10.67]

This simplified model estimates approximately 11 additional defects in future testing.

However, this is only a rough forecast. It does not prove that exactly 11 defects remain undetected. The actual number depends on the remaining test scope, test effectiveness, software changes, and defect severity.

4.8 Weaknesses of the Three Criteria

Criterion	**Main weakness**	**Example**
Test-case completion	Focuses on completion rather than risk	All tests may be executed while a critical defect remains open
Defect-trend stopping	Low defect discovery may be misleading	Testers may have run fewer tests during the final week
Risk-and-cost stopping	Depends on assumptions	Incorrect defect cost estimates may produce a misleading decision

4.10 Recommended Stopping Rule

For SureClaim, a combined stopping rule is recommended.

Testing should stop only when the following conditions are satisfied:

1. All planned critical and high-risk test cases are executed.
2. At least 100% of mandatory test cases are executed or formally approved exceptions are documented.
3. The overall pass rate is at least 95%.
4. No open critical defects remain.
5. All major defects have an approved resolution or documented business decision.
6. Requirement coverage is complete for all release-critical requirements.
7. Defect discovery is below the agreed threshold for two consecutive weeks.
8. Configuration and release evidence are complete.

9. The release manager and business owner approve the remaining risk.

At week 9, SureClaim does not satisfy these conditions because 60 planned tests remain unexecuted, the pass rate is 90.30%, and three critical defects remain open.

4.11 Judging the Statement: “Testing Is Never Complete; It Only Stops”

Testing cannot prove that software contains no defects. It can only provide evidence that the application has been tested to an agreed level and that the remaining risks are acceptable.

Therefore, the statement is reasonable. Testing stops when the agreed completion criteria are satisfied or when the organization makes a documented decision to accept the remaining risk.

The release manager should explain that stopping testing does not mean the application is completely free from defects. It means that the testing evidence and remaining risks have been reviewed and accepted by the responsible people.

4.12 Result

The three stopping criteria were applied to the week 9 data. The test-case completion and defect-trend criteria were not satisfied. The risk-and-cost analysis showed that unresolved critical defects create significant release risk.

4.13 Conclusion

A clear stopping criterion is necessary to control software testing and avoid premature release decisions. Test-case completion, defect trends, and risk-and-cost analysis provide different types of evidence.

For SureClaim, a combined stopping rule is recommended because it considers testing progress, defect severity, requirement coverage, and financial risk. Testing should not stop at week 9 while critical defects remain open.

TASK 5 – SOFTWARE CONFIGURATION MANAGEMENT: COMPARE CONFIGURATION CONTROL APPROACHES

5.1 Aim

To compare different software configuration management approaches and identify a suitable method to prevent incorrect configuration files from reaching the SureClaim production environment.

5.2 Introduction

Software Configuration Management (SCM) is the process of identifying, organizing, controlling, and tracking changes to software items throughout the development lifecycle.

A software project contains several configuration items, including source code, configuration files, build scripts, test scripts, test data, and documents. These items must be controlled carefully because an incorrect version can affect the behaviour of the application.

In the previous SureClaim release, a test copy of the fraud-threshold configuration file reached production. This incident shows that the organization needs stronger configuration control.

Three approaches are compared: shared-folder versioning, Git-based version control, and full SCM with baselines and formal change control.

5.3 Objectives

1. To understand software configuration management.
2. To prepare three configuration structures.
3. To perform sample threshold changes, bug fixes, and rollbacks.
4. To replay the incorrect configuration deployment incident.
5. To compare traceability, reproducibility, security, and effort.
6. To recommend a suitable SCM approach.

5.4 Configuration Items in SureClaim

The following items should be controlled:

Configuration item	Purpose

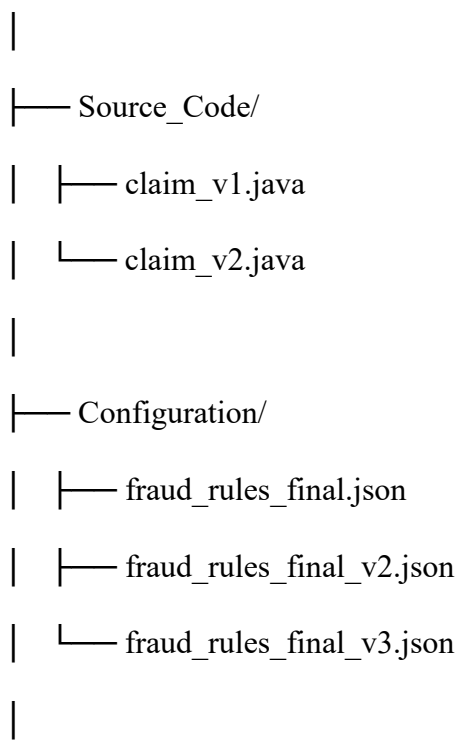
Source code	Contains application logic
fraud_rules.json	Stores fraud detection thresholds and rules
Premium tables	Stores premium-related values
Claim-limit tables	Defines claim limits
Build scripts	Controls application build activities
Test scripts	Contains automated and manual test definitions
Test data	Supports testing activities
Release notes	Records release information
Test reports	Provides testing evidence
Deployment scripts	Controls deployment activities

5.5 Approach A – Shared-Folder Versioning

Under this approach, files are stored in a shared folder with different names to identify versions.

Example:

SureClaim/



├── Test_Scripts/

│ └── claim_test.py

│

└── Release_Notes/

└── release_v2.txt

Sample Changes

Change 1 – Threshold Change

The fraud threshold is updated from 0.70 to 0.75. A new file is saved with a different version name.

Change 2 – Bug Fix

A developer corrects a claim validation error and saves a revised source file.

Change 3 – Rollback

If the latest version causes a problem, the team manually copies an earlier file into the active folder.

Advantages:

1. Simple to understand.
2. Does not require advanced tools.
3. Suitable for very small projects.
4. Easy to create manually.

Disadvantages:

1. Files may be overwritten.
2. Multiple copies can create confusion.
3. Change history is difficult to track.
4. Unauthorized changes may not be detected.
5. Rollback depends on manual file selection.

6. It is difficult to prove which file was deployed.

5.6 Approach B – Git-Based Version Control

Git stores changes in a repository and maintains a history of commits. Developers can use branches to work on different changes without directly modifying the release version.

Example structure:

```
SureClaim/
│
├── src/
│   └── claim_settlement.py
│
├── config/
│   ├── fraud_rules.json
│   ├── premium_table.csv
│   └── claim_limits.csv
│
├── tests/
│   └── test_claim.py
│
├── build/
│   └── build_script.sh
│
└── docs/
    └── release_notes.md
```

Sample Git Workflow

```
git init
```

git add .

git commit -m "Initial SureClaim baseline"

git checkout -b feature/fraud-threshold

After changing the threshold:

git add config/fraud_rules.json

git commit -m "Update fraud threshold"

A bug fix can be committed separately:

git add src/claim_settlement.py

git commit -m "Fix claim settlement validation"

A release tag can be created:

git tag -a v2.0 -m "SureClaim version 2.0 release"

A previous release can be restored by checking out the appropriate tag or by reverting the relevant change through an approved process.

Advantages:

1. Maintains change history.
2. Supports branching and merging.
3. Helps identify who changed a file.
4. Supports release tags.
5. Makes rollback easier.
6. Supports collaboration among developers.

Disadvantages:

1. Requires Git knowledge.
2. Incorrect branching practices can create conflicts.
3. A Git repository alone does not guarantee that the correct file is deployed.
4. Deployment permissions and release approval must be configured separately.

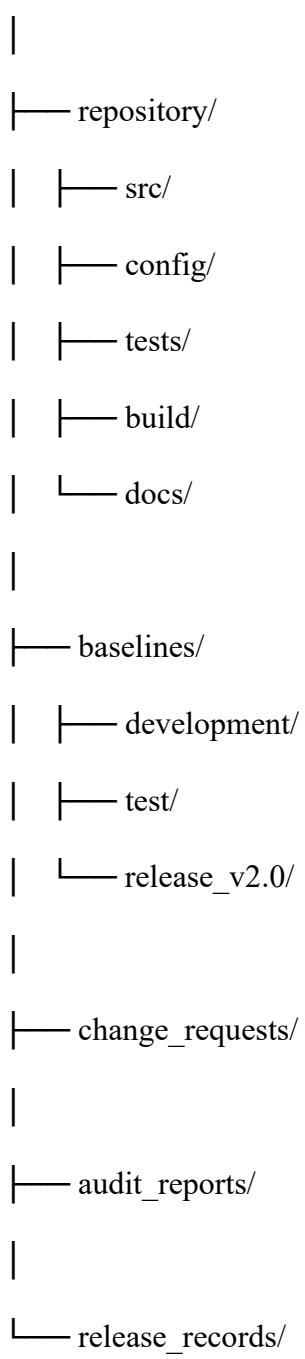
5.7 Approach C – Full SCM with Baselines and Change Control

Full SCM includes Git-based version control, formal baselines, approval processes, and automated configuration audits.

A baseline is an approved and fixed collection of configuration items for a particular release.

Example:

SureClaim/



Configuration Control Process

1. A developer submits a change request.
2. The change is reviewed.
3. The approved change is committed to Git.
4. Automated tests are executed.
5. A release baseline is created.
6. The deployment package is generated from the approved baseline.
7. The deployed files are compared with the baseline.
8. Audit evidence is stored.

Advantages:

1. Strong change traceability.
2. Supports reproducible releases.
3. Reduces unauthorized configuration changes.
4. Provides formal approval evidence.
5. Supports automatic configuration audits.
6. Improves release accountability.

Disadvantages:

1. Requires more administration.
2. Requires tool configuration.
3. Formal approvals may increase processing time.
4. Team members must follow the defined process.

5.8 Incident Replay: Wrong fraud_rules.json Deployment

The incident is replayed conceptually to compare the detection and recovery capabilities of the three approaches.

Assume that a test copy of fraud_rules.json is accidentally selected for production deployment.

Approach	How the error may be detected	Recovery method	Recovery estimate
Shared folder	Manual file inspection may identify the incorrect file	Manually copy the correct version	30–60 minutes
Git-based	Compare deployed commit or tag with the approved release	Restore the approved tagged version	10–20 minutes
Full SCM	Automated audit compares deployed hash with approved baseline	Redeploy the approved baseline and retain audit evidence	5–15 minutes

The recovery times are illustrative estimates for comparison. They are not measured results from an actual deployment exercise.

5.9 Change-History Comparison

Factor	Shared folder	Git	Full SCM
Change history	Manual	Commit history	Commit history and formal records
Version identification	File names	Commit IDs and tags	Baselines and tags
Rollback	Manual	Supported	Controlled and auditable
Approval	Informal	Pull request or review	Formal change approval
Unauthorized change prevention	Weak	Medium to strong with permissions	Strong with approval and audit
Release reproduction	Difficult	Good	Very good
Administrative effort	Low	Medium	High

5.10 Relationship Between Configuration Management and Testing

Configuration management is directly connected to software testing.

A test result is meaningful only when the tested software version and configuration are known.

For example, if the fraud detection tests are executed using a threshold of 0.70 but production uses an unapproved test threshold of 0.60, the test results may not represent the actual production behaviour.

The testing record should therefore identify:

- Source-code commit.
- Configuration-file version.
- Test-script version.
- Test-data version.
- Build number.
- Environment.
- Execution date.
- Test result.

Without this information, the team may not be able to reproduce a defect or confirm that the released application matches the tested application.

5.11 Items That Must Be Under Change Control

The following items should be placed under formal change control:

1. Source code.
2. fraud_rules.json.
3. Premium tables.
4. Claim-limit tables.
5. Build scripts.
6. Deployment scripts.

7. Test scripts.
8. Test data containing controlled or sensitive values.
9. Release notes.
10. Approved test and release baselines.

Configuration changes that affect fraud decisions, premium calculations, claim settlement, or payout limits should require formal review and approval.

5.12 Critique of Automatic AI Configuration Changes

An AI tool may suggest changes to fraud thresholds based on historical data. However, allowing it to directly modify production configuration files creates several risks.

1. The AI may recommend an incorrect threshold.
2. Historical data may contain bias or incomplete information.
3. The reason for a configuration change may not be clear.
4. Automatic changes may not follow business or regulatory approval.
5. A model may produce different recommendations under different conditions.
6. An unauthorized change may affect fraud detection and claim processing.

AI should therefore be used as a recommendation tool rather than as an unrestricted configuration administrator.

Every proposed change should be reviewed, tested, approved, committed to Git, and included in an approved baseline before deployment.

5.13 Final SCM Recommendation

Full SCM with baselines and change control is recommended for SureClaim.

The system handles fraud decisions and claim payments, so configuration mistakes can cause financial and operational problems. A formal SCM process provides better traceability and prevents an unapproved test configuration from reaching production.

Git should be used as the main version-control system, while formal approval and automated audits should be used for critical configuration items.

5.14 Result

Three configuration management approaches were compared. Their repository structures, change histories, incident-recovery methods, and control capabilities were examined.

5.15 Conclusion

Software configuration management is essential for maintaining consistency between the tested and released application. Shared-folder versioning is simple but provides weak control. Git improves version tracking and rollback, while full SCM provides stronger approval and audit capabilities.

For SureClaim, full SCM with Git, approved baselines, formal change control, and automated audits is the most suitable approach.

TASK 6 – TYPES OF REVIEWS: COMPARE REVIEW TECHNIQUES

6.1 Aim

To compare different software review techniques and identify the most suitable review method for important SureClaim work products.

6.2 Introduction

Software reviews are activities in which work products are examined to identify defects before they reach later stages of development or production.

Reviews may be performed on requirements, designs, source code, test cases, and configuration files. They help identify mistakes early and reduce the cost of correcting them.

The SureClaim platform contains important functions such as fraud detection and claim settlement. These functions should be reviewed carefully because errors may affect financial transactions and customer decisions.

Three review techniques are considered: peer desk-check, technical review, and formal inspection.

6.3 Objectives

1. To understand different review techniques.
2. To apply the techniques to the same sample work product.
3. To calculate defect detection percentage.
4. To calculate defects found per hour.
5. To compare the cost savings of early defect detection.
6. To understand human and organizational problems during reviews.
7. To evaluate the role of AI in software reviews.

6.4 Comparison of Review Techniques

A. Peer Desk-Check

A peer desk-check is an informal review in which one colleague reads a work product and provides comments to the author.

Advantages:

1. Easy to organize.
2. Requires limited preparation.
3. Suitable for small code changes.
4. Helps identify simple mistakes quickly.

Disadvantages:

1. May not identify complex defects.
2. Does not always maintain formal records.
3. Review quality depends on the individual reviewer.
4. Important requirements may be overlooked.

B. Technical Review

A technical review is conducted by a small group of technical reviewers. The reviewers examine the work product against requirements, design standards, and technical rules.

Advantages:

1. Provides multiple viewpoints.
2. Helps identify technical problems.
3. Supports discussion and clarification.
4. Records important review decisions.

Disadvantages:

1. Requires coordination.
2. May take more time than a desk-check.
3. Reviewers must understand the work product.
4. Discussions may move away from the main review objective.

C. Formal Inspection

A formal inspection follows a structured process with defined roles.

The main roles include:

- Moderator.
- Author.
- Reader.
- Recorder.
- Inspectors.

The inspection includes preparation, a review meeting, defect recording, and follow-up.

Advantages:

1. Provides a systematic review process.
2. Records identified defects.
3. Supports clear responsibilities.
4. Suitable for critical work products.
5. Provides measurable review results.

Disadvantages:

1. Requires more preparation.
2. Requires trained reviewers.
3. Takes more time.
4. May be difficult to arrange for every small change.

6.5 Sample Work Product

For comparison, consider a sample claim settlement function.

The function is responsible for:

1. Receiving a claim amount.
2. Checking claim eligibility.
3. Verifying the approved claim limit.

4. Checking fraud flags.
5. Calculating the final settlement amount.
6. Recording the payment status.
7. Returning the settlement result.

The review exercise assumes that eight defects have been deliberately seeded into the sample work product.

The defects are illustrative review targets and are not claims about actual SureClaim source code.

Sample Defect List

Defect ID	Defect description	Type
D1	Claim amount is not checked for negative values	Input validation
D2	Claim limit boundary is handled incorrectly	Boundary logic
D3	Fraud flag is ignored in one condition	Business logic
D4	Payment status is updated before confirmation	Transaction logic
D5	Null customer information is not handled	Exception handling
D6	Duplicate settlement request is not checked	Duplicate processing
D7	Incorrect rounding is used for payout calculation	Calculation
D8	Error message does not identify the validation failure	Usability

6.6 Applying the Three Review Techniques

For an illustrative comparison, assume that each review method identifies a different number of the eight seeded defects.

Review technique	Defects found	Time spent
Peer desk-check	3	1.5 hours
Technical review	6	3 hours

Formal inspection	7	4 hours

These are sample results for the assignment analysis. They must not be described as measured results from an actual inspection unless the review is performed.

6.7 Cost Savings from Early Defect Detection

The assignment provides these costs:

- Review: ₹2,500 per defect.
- System test: ₹12,000 per defect.
- After release: ₹60,000 per defect.

Savings compared with system testing:

[12000-2500=\boxed{₹9,500}]

Savings compared with post-release detection:

[60000-2500=\boxed{₹57,500}]

Review technique	**Defects found**	**Review-stage cost**	**Saving compared with system test**	**Saving compared with post-release detection**
Peer desk-check	3	₹7,500	₹28,500	₹1,72,500
Technical review	6	₹15,000	₹57,000	₹3,45,000
Formal inspection	7	₹17,500	₹66,500	₹4,02,500

These calculations compare the cost of finding the identified defects at different stages. They do not include reviewer salaries or meeting overhead because those costs were not supplied.

6.8 What Each Review Technique Finds Best

Review technique	**Suitable defect types**
Peer desk-check	Simple logic errors, coding style issues, missing comments

Technical review	Design problems, business logic mistakes, incorrect calculations
Formal inspection	Requirement ambiguities, complex logic errors, compliance and standards problems

A peer desk-check may miss complex defects because it usually involves one reviewer.

A technical review provides multiple viewpoints and is more suitable for complex design and code.

A formal inspection provides a structured method for reviewing critical work products and recording all findings.

6.9 Human and Organizational Problems

Author Defensiveness

Authors may feel that reviewers are criticizing their ability. The review should focus on the work product rather than the individual.

Reviewer Overload

Reviewers may have several responsibilities and may not have sufficient time. Review planning should consider reviewer availability.

Unprepared Reviewers

Reviewers who have not read the work product may waste time during the meeting. Preparation materials should be distributed before the review.

Personal Conflicts

Reviewers may disagree with each other. The moderator should keep discussions professional and focused on evidence.

Unclear Review Objectives

If the review purpose is not defined, the team may spend time discussing unrelated topics. The review scope and expected outcome should be stated clearly.

6.10 Role of AI as an Additional Reviewer

AI tools can help identify possible coding mistakes, missing conditions, duplicated logic, and unclear requirements.

However, AI-generated comments may be incorrect or incomplete. An AI tool may misunderstand business rules or recommend changes that are unsuitable for the insurance process.

AI should be used as an additional reviewer, not as a replacement for human review.

The human reviewer must verify every suggestion before making changes.

6.11 Recommended Review Rules for SureClaim

Work product	**Recommended review**
Small code change	Peer desk-check
Normal application design	Technical review
Fraud detection rules	Formal inspection
Claim settlement logic	Formal inspection
Release configuration	Formal review with audit
Test scripts	Peer or technical review
Requirements	Technical review, with formal inspection for critical requirements

6.12 Result

Three review techniques were compared using a sample claim settlement function containing eight seeded defects. The formal inspection had the highest assumed defect detection percentage at 87.5%, while the peer desk-check required the least time.

6.13 Conclusion

Software reviews help identify defects before they reach later testing stages. Peer desk-checks are suitable for simple changes, technical reviews are useful for complex technical work, and formal inspections are suitable for critical work products.

For SureClaim, formal inspections should be used for fraud rules and claim settlement logic because these features have a high business impact. AI can support the review process, but final decisions must remain with qualified reviewers.

TASK 7 – REVIEW PLANNING: COMPARE REVIEW PLANNING APPROACHES

7.1 Aim

To prepare and compare different software review plans and select a review strategy that covers important SureClaim work products within the available review effort.

7.2 Introduction

Review planning determines which work products should be reviewed, when the reviews should take place, who should participate, and how much time should be allocated.

The SureClaim project has 85 requirements, 12 design documents, and approximately 22 KLOC of code. The organization has limited review capacity and allows a maximum of 10% of project effort for reviews.

A suitable review plan should focus on high-risk work products without exceeding the available effort.

7.3 Objectives

1. To understand different review planning approaches.
2. To prepare three review plans.
3. To calculate review effort in person-hours.
4. To compare each plan with the 10% review limit.
5. To examine review entry and exit criteria.
6. To recommend a suitable review planning approach.

7.4 Comparison of Review Planning Approaches

A. Fixed Review Calendar

Under this approach, reviews are conducted every second Friday, regardless of the risk or size of the completed work product.

Advantages:

1. Easy to schedule.
2. Creates a regular review routine.

3. Helps reviewers plan their availability.

Disadvantages:

1. High-risk items may not receive enough attention.
2. Reviews may be scheduled before documents are ready.
3. Important changes may wait until the next review date.

B. Risk and Size-Driven Plan

This approach selects work products according to risk, complexity, and size.

High-risk items receive formal inspections, while lower-risk items receive desk-checks or technical reviews.

Advantages:

1. Focuses on high-risk work.
2. Uses review time more effectively.
3. Supports prioritization.
4. Can adapt to changes in project risk.

Disadvantages:

1. Requires accurate risk assessment.
2. Risk classification may involve judgment.
3. Some lower-risk items may receive limited review.

C. Gate-Based Plan

Reviews are connected to important project milestones.

Examples include:

- Requirements review before design.
- Design review before coding.
- Code review before test entry.

Advantages:

1. Finds problems early.
2. Prevents incomplete work from moving to the next stage.
3. Supports milestone control.
4. Provides clear approval points.

Disadvantages:

1. A delayed review can affect the schedule.
2. Work products may wait for formal approval.
3. Requires clear entry and exit criteria.

7.5 Plan A – Fixed Review Calendar

For this example, assume that a review takes place every second Friday during the 12-week release.

Six review meetings are planned.

Assumptions:

- Three reviewers participate.
- Each meeting lasts two hours.
- Each reviewer spends one hour preparing.
- The review meeting requires six person-hours per session.

Meeting effort:

[6\times3\times2=36\text{ hours}]

Preparation effort:

[6\times3\times1=18\text{ hours}]

Total effort:

[36+18=\boxed{54\text{ person-hours}}]

This plan has low effort, but it may not provide sufficient coverage of all 85 requirements, 12 design documents, and 22 KLOC of code.

7.7 Plan B – Risk and Size-Driven Plan

The following assumptions are used:

- Requirements: 85 items, 0.25 review hours per item, three reviewers.
- Design documents: 12 documents, one hour per document, three reviewers.
- Code: 22,000 lines reviewed at 150 lines per hour.
- Two reviewers participate in the code review estimate.

Requirement Review Effort

[85\times0.25\times3]

[=\boxed{63.75\text{ hours}}]

Design Review Effort

[12\times1\times3]

[=\boxed{36\text{ hours}}]

Code Review Effort

[\frac{22000}{150}=146.67\text{ review hours}]

For two reviewers:

[146.67\times2=\boxed{293.34\text{ person-hours}}]

Total Effort

[63.75+36+293.34]

[=\boxed{393.09\text{ person-hours}}]

Percentage of project effort:

[\frac{393.09}{5400}\times100]

[=\boxed{7.28%}]

This plan is within the 10% review limit.

7.8 Plan C – Gate-Based Plan

The gate-based approach connects reviews to project milestones.

Gate	Work product	Review type	Participants
Gate 1	Requirements	Technical review	Business analyst and reviewers
Gate 2	Design documents	Technical review or inspection	Developers and QA
Gate 3	Critical code	Formal inspection	Moderator, author, reader, recorder and inspectors
Gate 4	Release configuration	Formal configuration review	QA and configuration engineer

For comparison, use the same estimated review effort as Plan B:

Work product	Estimated effort
Requirements	63.75 hours
Design documents	36 hours
Code	293.34 hours
Total	393.09 hours

Percentage of project effort:

[\frac{393.09}{5400}\times100]

[=\boxed{7.28%}]

The gate-based plan is within the 10% limit.

7.9 Comparison of the Three Plans

Factor	Fixed calendar	Risk and size-driven	Gate-based

Review effort	54 hours under the stated assumptions	393.09 hours	393.09 hours
Risk coverage	Depends on meeting timing	High-risk items prioritized	Important milestone work products reviewed
Early defect detection	May be delayed	Good	Very good
Schedule flexibility	Low to medium	High	Medium
Resource planning	Easy	Requires risk analysis	Requires milestone coordination
Main limitation	May miss urgent work	Depends on risk assessment	Delays may block later activities

The fixed calendar has the lowest estimated effort, but it does not demonstrate complete review coverage. The risk-driven and gate-based plans provide more structured coverage within the review limit.

7.10 Review Entry and Exit Criteria

A review should not begin until the work product is ready.

Example: Fraud-Rule Design Review

Entry criteria	**Exit criteria**
Design document is complete	All major comments are recorded
Requirements are available	Critical defects are resolved
Reviewers are assigned	Remaining actions have owners
Relevant standards are available	Review decision is recorded
Review scope is defined	Approved version is identified

If a review begins before the entry criteria are satisfied, reviewers may waste time discussing incomplete information. Important issues may be missed, and the review may need to be repeated.

7.11 LLM-Generated Review Checklist Critique

An AI-generated checklist can provide useful starting points, but it must be checked for relevance.

A suitable checklist for SureClaim should include:

1. Are fraud thresholds defined clearly?
2. Are boundary values checked?
3. Are claim limits validated?
4. Are duplicate claim submissions handled?
5. Are payment failures handled safely?
6. Are customer details validated?
7. Are unauthorized configuration changes prevented?
8. Are error messages appropriate?
9. Are requirements traceable to test cases?
10. Are logging and audit requirements included?

A general checklist may be incomplete if it does not include insurance-specific business rules. The QA analyst should therefore review and customize the checklist before using it.

7.12 Recommended Review Plan

A combination of risk-driven and gate-based review planning is recommended.

- All requirements should receive an appropriate review.
- High-risk fraud and payout designs should receive formal inspection.
- Normal design documents should receive technical reviews.
- Critical code should be inspected before test entry.
- Configuration files should be reviewed before release baselines are created.

This approach places reviews early, prioritizes important work, and stays within the 10% project review limit under the stated effort assumptions.

7.13 Result

Three review plans were prepared and their effort was calculated. The risk-driven and gate-based plans required approximately 393.09 person-hours each, equal to 7.28% of the total project effort.

7.14 Conclusion

Review planning ensures that important work products are checked at the correct time. A fixed calendar is simple but may not focus on the highest risks. Risk-driven planning prioritizes complex and important items, while gate-based planning connects reviews to project milestones.

For SureClaim, combining risk-driven selection with milestone-based review gates provides the best balance of quality, effort, and schedule control.

TASK 8 – REVIEW REPORTING: COMPARE REVIEW REPORTING AND FOLLOW-UP APPROACHES

8.1 Aim

To compare different review reporting methods and establish an effective process for recording, tracking, and verifying review comments.

8.2 Introduction

Software reviews are useful only when identified defects are recorded and corrected. If review comments are given orally and not documented, important issues may be forgotten.

A review report provides information about the work product, reviewers, defects, review effort, decisions, and follow-up activities.

This task compares three approaches: minutes by email, formal review reports with defect logs, and tool-based reviews with tracked actions.

8.3 Objectives

1. To understand different review reporting approaches.
2. To prepare sample review records.
3. To classify at least ten review comments.
4. To calculate review metrics.
5. To compare follow-up traceability.
6. To understand how review data supports process improvement.
7. To recommend a suitable reporting approach.

8.4 Comparison of Review Reporting Approaches

A. Minutes by Email

Under this approach, the author prepares a summary of review comments and sends it to the reviewers.

Advantages:

1. Simple to prepare.

2. Requires no specialized tool.
3. Suitable for small reviews.
4. Provides a written summary.

Disadvantages:

1. Comments may be incomplete.
2. Actions may not have clear owners.
3. Follow-up may be forgotten.
4. Tracking several versions becomes difficult.

B. Formal Review Report with Defect Log

A formal report records the work product, reviewers, review effort, identified defects, severity, decision, and follow-up date.

Advantages:

1. Provides complete review evidence.
2. Supports defect classification.
3. Makes review decisions clear.
4. Helps track unresolved issues.
5. Supports quality audits.

Disadvantages:

1. Requires additional documentation effort.
2. Reports must be updated.
3. May be time-consuming for small changes.

C. Tool-Based Review with Tracked Actions

Under this approach, review comments are recorded in a pull request or review tool. Each action has an owner, status, and link to the related change.

Advantages:

1. Supports traceability.
2. Makes action ownership clear.
3. Provides an updated status.
4. Connects comments with source-code changes.
5. Helps maintain a review history.

Disadvantages:

1. Requires tool access and training.
2. Incorrect tool configuration may affect tracking.
3. Some business reviewers may prefer a simpler report.

8.5 Sample Work Product for Review

The selected work product is the fraud-rule design document for SureClaim.

The document describes:

- Fraud score thresholds.
- Claim risk categories.
- Manual review conditions.
- High-risk claim handling.
- Configuration control.
- Audit logging.
- Integration with the external claim system.

For this sample exercise, ten review comments are identified.

8.6 Ten Review Comments

ID	Review comment	Type	Severity
R1	Fraud threshold boundary is not clearly defined	Requirement ambiguity	Major

R2	Manual review condition is missing	Business logic	Major
R3	Duplicate claim submission is not addressed	Functional	Major
R4	Configuration file approval is not described	Configuration	Major
R5	Error handling for unavailable fraud service is missing	Exception handling	Major
R6	Audit log requirements are incomplete	Security/audit	Major
R7	Variable names are inconsistent	Coding standard	Minor
R8	One section contains an unclear description	Documentation	Minor
R9	The diagram does not show all data flows	Design	Minor
R10	A reference to an outdated requirement is present	Traceability	Minor

There are six major defects and four minor defects in this sample.

Major defect percentage:

[\frac{6}{10}\times100]

[=\boxed{60%}]

8.7 Format A – Minutes by Email

Subject: SureClaim Fraud-Rule Design Review – Meeting Minutes

The review was conducted to examine the fraud-rule design document.

The main observations were:

1. Fraud threshold boundaries require clarification.
2. Manual review conditions must be included.
3. Duplicate claim submission handling must be documented.
4. Configuration approval steps must be added.
5. Failure handling for the external fraud service must be explained.
6. Audit logging requirements must be improved.

7. Naming conventions should be standardized.
8. Unclear documentation should be corrected.
9. The design diagram should be updated.
10. Outdated requirement references should be replaced.

The author is requested to update the document and share the revised version with the reviewers.

Limitation: The email summary does not automatically track the status of each comment. Separate follow-up is required.

8.8 Format B – Formal Review Report

Review Title: SureClaim Fraud-Rule Design Review

Work Product: Fraud-rule design document

Review Type: Technical review

Participants: Author, moderator, business analyst, developer, QA analyst

Review Purpose: Identify requirement, design, configuration, and audit-related problems.

ID	Defect	Type	Severity	Action required
R1	Threshold boundary unclear	Requirement	Major	Define boundary values
R2	Manual review missing	Business logic	Major	Add manual review rule
R3	Duplicate claim not addressed	Functional	Major	Add duplicate check
R4	Configuration approval missing	Configuration	Major	Define approval process
R5	Service failure handling missing	Exception	Major	Define fallback behaviour
R6	Audit log incomplete	Audit	Major	Add required audit fields
R7	Naming inconsistent	Standard	Minor	Standardize names

R8	Unclear description	Documentation	Minor	Rewrite section
R9	Incomplete diagram	Design	Minor	Update diagram
R10	Outdated reference	Traceability	Minor	Update requirement link

Review Decision: Accept with changes, subject to correction of all major comments and verification of the revised document.

Follow-up Date: Before the next design approval meeting.

8.9 Format C – Tool-Based Review

A tool-based review can be implemented through Git pull requests, JIRA, or another review-management tool.

Each comment should contain:

- Review ID.
- Comment description.
- Defect type.
- Severity.
- Assigned owner.
- Due date.
- Status.
- Link to the related change.

Example:

ID	Owner	Status	Verification
R1	Business analyst	Open	Pending
R2	Developer	In progress	Pending
R3	Developer	Open	Pending
R4	Configuration engineer	In progress	Pending

R5	Developer	Open	Pending
R6	QA analyst	Open	Pending
R7	Developer	Closed	Verified
R8	Author	Closed	Verified
R9	Designer	In progress	Pending
R10	Business analyst	Closed	Verified

This table is an illustrative review-tracking example. Actual completion must be confirmed by the assigned owners.

8.10 Review Metrics

Assume that the sample review required four hours.

Defects Found per Hour

[\frac{10}{4}=\boxed{2.5\text{ defects per hour}}]

Defects per Page

Assume the reviewed design document contains eight pages.

[\frac{10}{8}=\boxed{1.25\text{ defects per page}}]

Review Effort

[\boxed{4\text{ person-hours}}]

This assumes that the total effort of all reviewers is four person-hours. If several reviewers each spend four hours, total person-hours must be increased accordingly.

Percentage of Major Defects

[\frac{6}{10}\times100]

[=\boxed{60%}]

8.11 Follow-Up Traceability After Two Weeks

For comparison, assume the following follow-up results:

Reporting approach	Actions traceable after two weeks	Actions verified	Limitation
Email minutes	6 of 10	4 of 10	Manual follow-up required
Formal report	9 of 10	8 of 10	Report updates are required
Tool-based tracking	10 of 10	9 of 10	Depends on correct tool usage

These values are illustrative assumptions for comparing the reporting methods.

Tool-based tracking provides the strongest action traceability because each comment has an owner and status.

8.12 Using Review Data for Process Improvement

Review records can help the organization identify repeated problems.

For example, if requirement ambiguity appears frequently, the organization may improve requirement-writing guidelines.

If configuration defects are common, the team may improve change-control procedures.

If audit-related comments are frequently raised, audit requirements should be introduced earlier during design.

The most frequent defect types can be identified through a simple classification table or chart.

8.13 Risks of Reporting Individual Performance

Review data should be used to improve the software process, not to unfairly rank individual employees.

For example, a reviewer who reports fewer defects may have reviewed a simple work product, while another reviewer may have reviewed a complex and high-risk component.

Similarly, an author who receives more comments may have worked on a more complicated feature.

Therefore, review data should be interpreted in context and should not be used as the only measure of individual performance.

8.14 AI Review Summary Reliability

AI tools can summarize review comments and prepare reports. However, they may omit important comments, change the severity of a defect, or misunderstand technical terms.

The AI-generated summary should be compared with the original review record.

The QA analyst must verify:

1. All comments are included.
2. Severity is correctly recorded.
3. Owners are correctly assigned.
4. Decisions are accurately represented.
5. No unsupported claims are added.
6. Follow-up dates are correct.

AI can reduce documentation effort, but the original review records must remain the source of truth.

8.15 Final Recommendation

The tool-based review approach is recommended for regular SureClaim development work because it provides strong traceability and action ownership.

Formal review reports should be maintained for critical work products, including fraud rules, settlement logic, and release configurations.

Email minutes may be used for informal communication but should not be the only record of important review actions.

Decision Rule: Accept with Changes or Re-review

Accept with changes may be used when:

- Remaining comments are clearly documented.
- Major corrections can be verified without repeating the full review.

- No unresolved critical issue affects the design.

Re-review should be required when:

- Major design changes are introduced.
- Critical defects are identified.
- Requirements change significantly.
- The corrected work product requires another full technical evaluation.

8.16 Result

Three review reporting formats were prepared and compared. Ten review comments were classified, and review metrics were calculated. The tool-based approach provided the strongest sample follow-up traceability.

8.17 Conclusion

Review reporting ensures that identified defects are not forgotten and that corrective actions are completed. Email minutes are simple, formal reports provide detailed evidence, and tool-based reviews support action tracking.

For SureClaim, tool-based tracking supported by formal reports for critical work products is recommended. This approach improves traceability, accountability, and review quality.

TASK 9 – INTEGRATED QUALITY MONITORING AND CONTROL: COMPARE COMPLETE CONFIGURATIONS

9.1 Aim

To combine the measurement, reporting, monitoring, stopping, configuration management, and review practices selected in the previous tasks into one integrated quality monitoring and control process for SureClaim v2.0.

9.2 Introduction

Software quality cannot be controlled through a single testing activity. Measurements, reports, defect management, configuration control, reviews, and release decisions must work together.

The previous tasks examined these activities separately. This task combines them into complete quality control configurations and compares their effectiveness.

Three configurations are considered:

1. Minimal configuration.
2. Balanced configuration.
3. Comprehensive configuration.

An additional AI-assisted configuration is also evaluated.

9.3 Objectives

1. To combine the selected practices from Tasks 1 to 8.
2. To compare three complete quality control configurations.
3. To develop a prototype for calculating weekly metrics.
4. To apply colour rules and stopping criteria.
5. To prepare a block diagram.
6. To prepare a design decision table.
7. To compare quality control requirements across different application types.
8. To evaluate an AI-assisted configuration.

9.4 Configuration A – Minimal Configuration

The minimal configuration uses:

- Plan-versus-actual measurements.
- Weekly narrative email reports.
- Shared-folder versioning.
- Informal reviews.
- Basic test completion criteria.

Advantages:

1. Low initial setup effort.
2. Easy to understand.
3. Requires fewer tools.
4. Suitable for small projects.

Disadvantages:

1. Limited audit evidence.
2. Weak configuration control.
3. Less reliable defect tracking.
4. Greater dependence on manual reporting.
5. Critical risks may not be identified early.

9.5 Configuration B – Balanced Configuration

The balanced configuration uses:

- Defect and coverage metrics.
- Weekly RAG status reports.
- Git-based version control with release tags.
- Risk-based reviews.
- Defined stopping criteria.

- Defect tracking through JIRA or Bugzilla.

Advantages:

1. Provides useful quality measurements.
2. Supports traceable configuration changes.
3. Focuses review effort on high-risk areas.
4. Provides clear release criteria.
5. Requires moderate operating effort.

Disadvantages:

1. Some information may still require manual collection.
2. Configuration audits may not be fully automatic.
3. Reporting depends on correct tool usage.

9.6 Configuration C – Comprehensive Configuration

The comprehensive configuration uses:

- Live dashboards.
- Automatic configuration audits.
- Formal inspections of critical work products.
- Tool-based review tracking.
- Risk-and-cost stopping criteria.
- Automated metric calculations.
- Approved configuration baselines.

Advantages:

1. Strong traceability.
2. Early warning of problems.
3. Better audit evidence.

4. Strong configuration control.
5. Supports informed release decisions.

Disadvantages:

1. Higher initial setup effort.
2. Requires tool integration.
3. Needs trained team members.
4. More administrative control is required.

9.7 Prototype Design

The prototype reads the weekly testing data and calculates the final measurement set.

The prototype should calculate:

1. Test execution percentage.
2. Test pass rate.
3. Schedule gap.
4. Open defect count.
5. Defect closure rate.
6. Defect density.
7. Requirement coverage when available.
8. RAG status.
9. Stopping criteria.

Sample Python Prototype

The following program uses the weekly data provided in the assignment. It calculates the metrics and generates charts.

```
import pandas as pd

import matplotlib.pyplot as plt
```

```
data = {
    "Week": [1, 2, 3, 4, 5, 6, 7, 8, 9],
    "Planned": [120, 300, 520, 740, 960, 1150, 1280, 1360, 1400],
    "Executed": [100, 240, 410, 560, 720, 900, 1080, 1230, 1340],
    "Passed": [88, 200, 340, 450, 590, 750, 920, 1075, 1210],
    "Found": [15, 42, 80, 130, 178, 214, 238, 252, 260],
    "Closed": [5, 20, 46, 78, 118, 156, 190, 224, 240]
}
df = pd.DataFrame(data
df["Execution_Percent"] = (
    df["Executed"] / df["Planned"] * 100
)
df["Pass_Rate"] = (
    df["Passed"] / df["Executed"] * 100
)
df["Schedule_Gap"] = (
    df["Planned"] - df["Executed"]
)
df["Open_Defects"] = (
    df["Found"] - df["Closed"]
)
df["Closure_Rate"] = (
    df["Closed"] / df["Found"] * 100
)df["Defect_Density"] = (
```

```
    df["Found"] / 22
)
df["Unexecuted_Percent"] = (
    df["Schedule_Gap"] / df["Planned"] * 100
)
def colour_status(row):
    if row["Unexecuted_Percent"] > 15:
        return "RED"
    elif row["Unexecuted_Percent"] > 5:
        return "AMBER"
    else:
        return "GREEN"
df["Schedule_Status"] = df.apply(
    colour_status, axis=1
)
print(df.round(2))
plt.figure()
plt.plot(df["Week"], df["Planned"], marker="o",
         label="Planned")
plt.plot(df["Week"], df["Executed"], marker="o",
         label="Executed")
plt.xlabel("Week")
plt.ylabel("Test Cases")
plt.title("Planned vs Executed Tests")
```

```
plt.legend()
plt.grid()
plt.show()
plt.figure()
plt.plot(df["Week"], df["Found"], marker="o",
        label="Defects Found")
plt.plot(df["Week"], df["Closed"], marker="o",
        label="Defects Closed")
plt.xlabel("Week")
plt.ylabel("Defects")
plt.title("Defects Found vs Closed")
plt.legend()
plt.grid()
plt.show()
plt.figure()
plt.plot(df["Week"], df["Open_Defects"],
        marker="o")
plt.xlabel("Week")
plt.ylabel("Open Defects")
plt.title("Open Defect Trend")
plt.grid()
plt.show()
```

Expected Metric Calculations

The following values are calculated directly from the assignment's cumulative data.

Week	Execution %	Pass rate %	Gap	Open defects	Closure rate %	Defect density
1	83.33	88.00	20	10	33.33	0.68
2	80.00	83.33	60	22	47.62	1.91
3	78.85	82.93	110	34	57.50	3.64
4	75.68	80.36	180	52	60.00	5.91
5	75.00	81.94	240	60	66.29	8.09
6	78.26	83.33	250	58	72.90	9.73
7	84.38	85.19	200	48	79.83	10.82
8	90.44	87.40	130	28	88.89	11.45
9	95.71	90.30	60	20	92.31	11.82

These results show that the schedule gap decreased after week 6. The defect backlog also reduced from 60 at week 5 to 20 at week 9.

However, defect density exceeded the target of 10 defects per KLOC from week 7 onward.

9.8 Stopping Criteria in the Prototype

The prototype can apply the following basic checks:

```
for index, row in df.iterrows():
    if row["Execution_Percent"] >= 100:
        execution_complete = True
    else:
        Execution_complete = False
    if row["Pass_Rate"] >= 95:
        pass_target_met = True
    else:
        pass_target_met = False
```

```
if row["Open_Defects"] == 0:
    defect_backlog_clear = True
else:
    defect_backlog_clear = False
print(
    "Week:", row["Week"],
    "Execution complete:", execution_complete,
    "Pass target met:", pass_target_met,
    "Defect backlog clear:", defect_backlog_clear
)
```

The above code checks basic test completion conditions. For a complete release decision, the actual system must also check critical defect severity, requirement coverage, configuration audit results, and review completion.

The weekly data does not contain severity information for every week. Therefore, the three critical open defects supplied at week 9 must be checked separately.

9.9 Comparison of the Three Configurations

Evaluation factor	**Minimal**	**Balanced**	**Comprehensive**
Early warning	Low	Good	Very good
Operating cost	Low	Medium	High
Review control	Informal	Risk-based	Formal inspections
Configuration control	Shared folder	Git and tags	Git, baselines and audits
Audit evidence	Limited	Good	Strong
Release decision accuracy	Limited	Good	Very good
Manual effort	High	Medium	Lower after automation

The minimal configuration is less expensive but has weaker traceability and audit evidence.

The balanced configuration provides useful control without excessive complexity.

The comprehensive configuration provides the strongest evidence and automation but requires additional setup and maintenance.

9.10 Comparing Predictions with Prototype Results

Some predictions from earlier tasks can be checked against the weekly data.

Prediction	Evidence	Evaluation
Schedule gap would increase initially	Gap reached 250 by week 6	Supported
Schedule progress would improve later	Gap reduced to 60 by week 9	Supported
Defect backlog would decrease after week 5	Open defects fell from 60 to 20	Supported
Defect density would remain below 10	It reached 11.82 by week 9	Contradicted
High execution percentage would guarantee release readiness	Three critical defects remained open	Contradicted

The data shows that progress and quality do not always improve together. A project may execute most planned tests while still having serious unresolved defects.

9.11 Integrated Quality Control Block Diagram

The integrated process can be represented as follows:

PROJECT REQUIREMENTS

|

v

TEST PLANNING

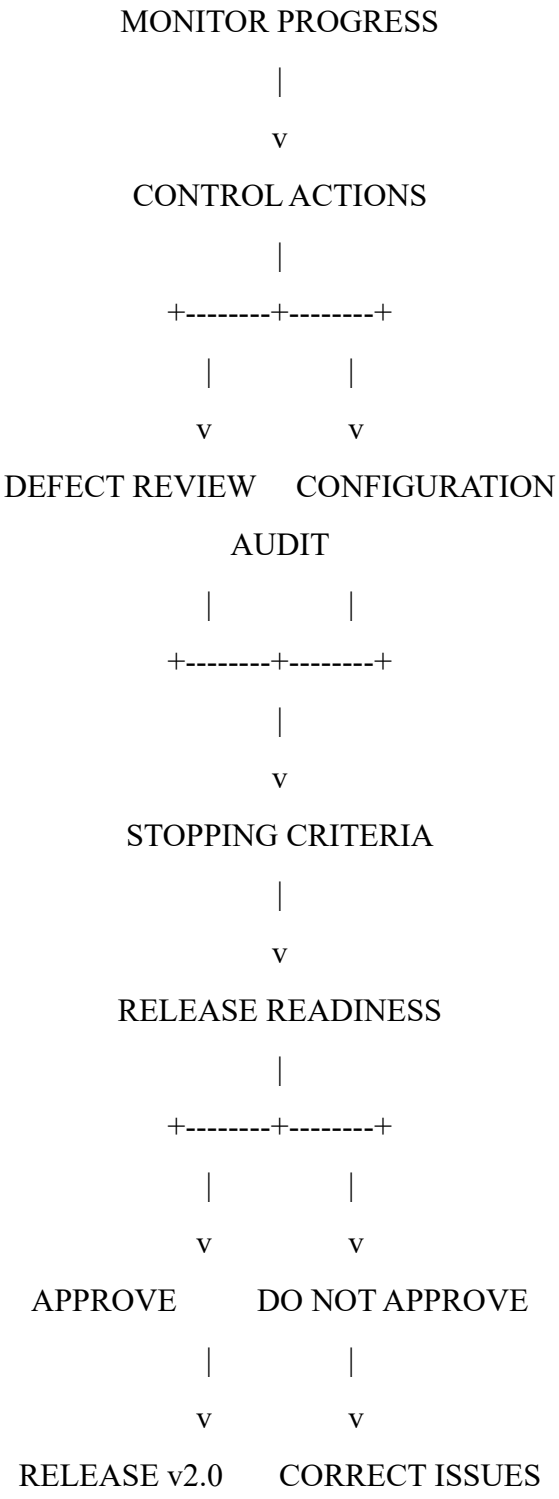
|

v

TEST EXECUTION

|

v



Explanation of the Diagram

The process begins with project requirements and test planning. Test cases are executed, and measurements are collected. The results are reported to the release manager and monitored against the agreed targets.

When a problem is identified, the team takes corrective action. Defect reviews and configuration audits are performed to ensure that the software and its configuration are controlled.

The stopping criteria are then evaluated. If the required quality conditions are satisfied, the release can be approved. Otherwise, the identified issues must be corrected before release.

9.12 Design Decision Table

Decision	**Justification (Evidence)**	**What If Not?**
Use eight core metrics	Provides schedule, quality and coverage information	Important problems may remain hidden
Use weekly RAG reports	Makes management status easy to understand	Management may not notice delays
Use daily dashboards	Supports quick team-level monitoring	Defect and test status may become outdated
Use Git with tags	Supports version history and release reproduction	Incorrect files may be deployed
Use formal baselines	Identifies approved release items	Tested and released configurations may differ
Use automatic configuration audits	Checks deployed files against approved baselines	Configuration errors may remain unnoticed

9.13 Adapting the Practice to Other Releases

Different software applications have different risks. Therefore, testing measurements, stopping criteria, and review intensity should be adapted to the application.

Factor	**SureClaim**	**ATM-switch upgrade**	**Hospital pharmacy system**	**Mobile game**
Main risk	Incorrect claims and fraud decisions	Transaction failure and financial loss	Medication and patient safety	Gameplay and user experience

Important metrics	Defect severity, coverage, payout accuracy	Transaction success, latency, recovery	Prescription accuracy, safety tests, audit	Crash rate, frame rate, usability
Stopping criteria	No open critical defects and approved risk	Strict transaction and recovery requirements	No unresolved safety-critical defects	Stability and agreed experience targets
Configuration control	Formal baselines and fraud-rule control	Strict release and rollback control	Strong audit and approval control	Version control and release testing

ATM-Switch Upgrade

An ATM-switch upgrade handles financial transactions. Testing should focus on transaction accuracy, availability, security, response time, and recovery from failures.

Critical transaction failures should prevent release.

Hospital Pharmacy System

A hospital pharmacy system may affect medication ordering, dispensing, and patient safety. Testing should focus on correctness, access control, audit trails, and safety-related requirements.

Any unresolved safety-critical defect should prevent release.

Mobile Game

A mobile game may focus more on user experience, compatibility, performance, crash rate, and gameplay behaviour. Review and stopping criteria can be adjusted according to the game's risk and business requirements.

9.14 Limitations of Weekly Cumulative Data

The weekly tracking data provides cumulative values. It does not show every detail of individual test execution.

For example:

1. It does not identify which test cases were executed again after fixes.
2. It does not identify the time taken to execute each test.
3. It does not show which test cases were blocked.
4. It does not provide weekly defect severity.
5. It does not identify the exact cause of schedule delays.
6. It does not show whether a passed test used the final configuration.
7. It does not show how many defects were reopened.

A test case may pass before a software fix and fail after the fix. Cumulative data may not clearly show this repeated activity.

9.15 Final Configuration Recommendation

The comprehensive configuration is recommended for critical SureClaim functions because it provides the strongest configuration control, review evidence, and release monitoring.

However, the balanced configuration can be used as the operational foundation, with comprehensive controls added for fraud rules, claim settlement, payout, and production configuration. This approach provides strong control without introducing unnecessary complexity into every activity.

9.16 Result

Three quality control configurations were compared and a prototype was prepared to calculate weekly metrics and generate charts. A block diagram and decision table were also prepared. The analysis identified the advantages and limitations of each configuration.

9.17 Conclusion

Integrated quality monitoring combines measurements, reporting, control actions, reviews, configuration audits, and release decisions. The weekly data showed that test execution improved significantly after week 6, but defect density exceeded its target and critical defects remained open at week 9.

A balanced approach supported by formal configuration control, risk-based reviews, automatic metrics, and clear stopping criteria is recommended for SureClaim.

TASK 10 – RE-ANALYZE AND ADAPT THE QUALITY CONTROL PRACTICE

10.1 Aim

To re-evaluate the quality control practice selected for SureClaim and adapt it to the changed requirements of fortnightly releases, continuous delivery, external AI fraud scoring, UPI payouts, outsourced development, and regulatory evidence requirements.

10.2 Introduction

Software development environments change over time. A quality control process that works for a 12-week release may not be suitable for a continuous delivery environment with releases every two weeks.

SureClaim now has several new requirements:

- Fortnightly releases.
- Continuous delivery.
- Instant claim settlement through UPI within 30 minutes.
- An external AI fraud-scoring service.
- A development team of 24 people.
- Three outsourced vendors located in different cities.
- Mandatory regulatory evidence for every release.
- A maximum quality-control overhead of 8% per two-week cycle.

These changes require the original quality control process to be reviewed and adapted.

10.3 Objectives

1. To identify which original assumptions are no longer valid.
2. To compare three quality control adaptation strategies.
3. To calculate the new two-week effort limit.
4. To evaluate the original review and reporting effort.
5. To prepare a sprint-sized prototype.

10.4 Changed Project Conditions

Area	Original condition	New condition
Release duration	12 weeks	2 weeks
Delivery method	Planned release	Continuous delivery
Payment	Claim settlement payout	UPI payout within 30 minutes
Fraud detection	Internal fraud rules	External AI fraud-scoring service
Team size	15 people	24 people
Team location	Mainly one project team	Three outsourced vendors in different cities
Audit	Project-level evidence	Evidence for every release
Overhead limit	10% for reviews	8% for the complete quality-control overhead

The original practice cannot be used unchanged because the new release cycle is shorter and the regulatory evidence requirement is stronger.

10.5 Identifying Invalid Original Assumptions

1. Release Duration

The original practice assumed a 12-week release. The new process delivers every two weeks.

A long reporting cycle is no longer suitable because problems must be identified within each short release cycle.

2. Team Location

The original process assumed a coordinated project team. The new team includes outsourced vendors in different cities.

Therefore, review actions, configuration changes, and release approvals must be recorded in shared tools.

3. Reporting Frequency

Weekly reports may be too slow for continuous delivery. The new process needs automated pipeline status and release-level reports.

4. Review Capacity

Reviews must take place more frequently. They should be smaller, focused, and connected to pull requests.

5. Configuration Audit

The original approach used release-level configuration control. The new process requires evidence for every release and automatic checks before deployment.

6. External AI Fraud Service

The system now depends on an external AI service. Testing must include integration failures, response time, unavailable service handling, and changes in external service behaviour.

7. UPI Payout

The payment process requires additional testing of transaction status, duplicate requests, time limits, failed transactions, and reconciliation.

10.6 Strategy A – Retain and Scale Down

Under this approach, the organization keeps the original quality control practice but reduces the number of metrics and shortens reviews.

The process includes:

- A smaller metric set.
- Shorter reviews.
- A report at the end of each release.
- Git-based configuration control.
- Basic release evidence.

Advantages:

1. Requires fewer process changes.
2. Easy for the existing team to adopt.
3. Lower initial implementation effort.
4. Retains familiar procedures.

Disadvantages:

1. May not provide enough evidence for every release.
2. Manual reporting may become slow.
3. Cross-vendor traceability may be difficult.
4. Configuration audits may remain incomplete.

10.7 Strategy B – Modify with Pipeline Gates and Lightweight Reviews

This approach adapts the original practice by introducing automated quality gates into the development pipeline.

The process includes:

1. Sprint-level testing metrics.
2. Automated build verification.
3. Pull-request reviews.
4. Test execution gates.
5. Defect severity checks.
6. Configuration validation.
7. Tool-generated release reports.

Advantages:

1. Supports frequent releases.
2. Provides early warning.
3. Reduces manual reporting.
4. Supports collaboration across locations.
5. Provides useful release evidence.

Disadvantages:

1. Requires pipeline setup.
2. Incorrect quality gates may block valid releases.
3. Requires training and tool maintenance.

4. Some business decisions still require human approval.

10.8 Strategy C – Redesign with Quality-as-Code

Quality-as-code means that quality rules, review requirements, configuration policies, and release conditions are maintained as version-controlled files.

The pipeline automatically checks these rules before a release.

Important controls include:

- Version-controlled quality gates.
- Signed builds.
- Approved configuration files.
- Automated test evidence.
- Pull-request review rules.
- Release audit records.
- Automatic evidence generation.

Advantages:

1. Strong auditability.
2. Consistent release control.
3. Supports frequent delivery.
4. Reduces manual configuration errors.
5. Provides evidence for every release.

Disadvantages:

1. Requires higher initial setup effort.
2. Needs experienced pipeline administrators.
3. Rules must be maintained carefully.
4. Incorrect rules may affect every release.

10.11 Example of an Adapted Two-Week Effort Budget

The following is a proposed effort budget. These are planning allocations, not actual measured effort.

Quality activity	**Planned person-hours**
Lightweight reviews	30
Automated testing and quality-gate maintenance	30
Configuration audit	15
Test and defect reporting	10
Release evidence preparation	15
Risk and release review	10
Total	110

10.12 Recommended Adaptation Decision

<ins>Decision: MODIFY the original practice using pipeline gates and lightweight reviews, while introducing selected quality-as-code controls for critical functions.</ins>

A complete immediate redesign may require significant setup effort. However, the new requirements are too demanding for the original manual approach.

Therefore, the organization should modify the current practice in stages.

The revised practice should include:

1. Automated test execution in the pipeline.
2. Pull-request reviews for code changes.
3. Mandatory approval for fraud-rule changes.
4. Signed and traceable release builds.
5. Automated configuration audits.

10.13 Revised Quality Control Process

Step 1 – Requirement and Change Review

Every change should be linked to an approved requirement or work item. Critical changes must receive review before implementation.

Step 2 – Development and Pull-Request Review

Developers should create separate branches and submit pull requests. The required reviewers must examine the changes before merging.

Step 3 – Automated Build and Test

The pipeline should execute build verification, unit tests, integration tests, and relevant regression tests.

Step 4 – Quality Gates

The pipeline should check:

- Test execution status.
- Critical test failures.
- Code review approval.
- Required configuration versions.
- Security checks.
- Build integrity.

Step 5 – Configuration Audit

The deployed configuration should be compared with the approved configuration baseline.

Step 6 – Release Evidence Generation

The system should generate a release record containing:

- Build identifier.
- Source-code commit.
- Configuration version.
- Test execution results.
- Defect summary.

- Review approvals.

Step 7 – Release Approval

The release manager should approve the release only after mandatory quality gates and regulatory evidence requirements are satisfied.

10.14 Testing Requirements for UPI Payout

The new instant claim settlement feature introduces additional risks.

Testing should include:

1. Successful UPI payout.
2. Invalid payment details.
3. Payment service unavailable.
4. Delayed payment response.
5. Duplicate payout request.
6. Claim amount validation.
7. Reconciliation between claim and payment records.
8. Verification of the 30-minute settlement requirement.

A failed payment should not be recorded as successful. Duplicate requests should be handled safely, and transaction records should remain traceable.

10.15 Testing Requirements for External AI Fraud Scoring

The external AI fraud-scoring service introduces dependency and integration risks.

Testing should include:

1. Successful service response.
2. Service unavailable.
3. Invalid response format.
4. Slow response.
5. Missing fraud score.

6. Unexpected score values.
7. Authentication failure.
8. Changes in the external service version.
9. Logging and traceability.
10. Correct handling of uncertain results.

The system should define what happens when the AI service is unavailable. The application should not silently approve or reject claims without following the approved business rules.

10.22 Result

The original quality control practice was re-evaluated under the new fortnightly release conditions. The two-week project effort was calculated as 1,440 person-hours, and the 8% overhead limit was calculated as 115.2 person-hours.

An example quality-control budget of 110 person-hours was prepared, representing 7.64% of project effort. Three adaptation strategies were compared, and a modified pipeline-based approach was recommended.

10.23 Conclusion

The new SureClaim requirements introduce a faster release cycle, external service dependencies, outsourced development, instant UPI payout, and stricter regulatory evidence requirements.

The original 12-week quality control process cannot be used unchanged. A modified approach with automated pipeline gates, lightweight reviews, configuration audits, and release-level evidence is more suitable.

Quality-as-code should be introduced for critical configuration and release policies. AI tools may support reporting and review activities, but all compliance evidence must be verified against the original records.

OVERALL DISCUSSION AND ANALYSIS

The completion of all ten tasks provided a detailed understanding of software testing measurement, monitoring, control, configuration management, review practices, and release quality management.

The SureClaim Motor Insurance Claims Platform v2.0 was used as the main application throughout the assignment. The platform includes important functions such as mobile claim intake, photo upload, automated damage estimation, fraud detection, claim settlement, and garage-partner services. Since the application handles customer information and financial activities, maintaining software quality is essential.

Task 1 focused on selecting and calculating software testing measurements. The analysis demonstrated that schedule progress, defect information, and test coverage provide different types of evidence. The week 5 and week 9 calculations showed that testing progress improved over time. However, defect density exceeded the specified target by week 9, showing that test completion alone is not sufficient to determine software quality.

Task 2 explained how testing status can be communicated through different reporting formats. The RAG summary provided a quick understanding of project health, the metric-chart pack showed testing trends, and the daily dashboard supported team-level monitoring. The comparison demonstrated that reporting should be selected according to the audience and the information required.

Task 3 examined how a delayed testing schedule can be controlled. The weekly data showed that the schedule gap reached 250 tests in week 6. Different approaches were considered, including adding resources, reducing low-risk scope, and removing the actual causes of delay. The analysis showed that identifying and correcting the root causes of delays is important for achieving sustainable improvement.

Task 4 focused on the criteria used to decide when testing can stop. The analysis showed that test execution, pass rate, defect trends, and financial risks must be considered together. SureClaim could not satisfy the proposed stopping conditions at week 9 because tests remained unexecuted and critical defects were still open.

Task 5 explained the importance of software configuration management. The incorrect deployment of the fraud-threshold configuration file demonstrated why configuration items

must be controlled. Shared-folder versioning provides limited protection, while Git and formal baselines offer better traceability and release reproduction.

Task 6 compared peer desk-checks, technical reviews, and formal inspections. The analysis showed that different review methods are suitable for different work products. Small changes may require simple peer reviews, while fraud rules and claim settlement logic require more structured reviews.

Task 7 focused on planning reviews within the available project effort. The calculations showed that risk-driven and gate-based review plans can provide stronger coverage while remaining within the 10% review limit under the stated assumptions. The task also demonstrated the importance of entry and exit criteria.

Task 8 examined how review comments should be recorded and followed up. Email minutes, formal reports, and tool-based review tracking were compared. The analysis showed that assigning owners and maintaining action statuses improves accountability and prevents important review comments from being forgotten.

Task 9 combined the previous activities into complete quality control configurations. The prototype calculated weekly metrics and generated charts from the supplied tracking data. The analysis showed that testing progress improved after week 6, while defect density continued to increase. This demonstrated why several quality measurements are needed for a reliable release decision.

Task 10 considered the changed project environment. The release cycle was reduced to two weeks, the team expanded, external services were introduced, and regulatory evidence became mandatory for every release. The task showed that the original quality control practice must be adapted to support continuous delivery and stronger audit requirements.

Overall, the ten tasks demonstrated that software quality depends on a combination of measurement, communication, monitoring, corrective action, configuration control, review, and release approval. These activities must be connected so that the organization can identify problems early and make decisions using reliable evidence.

LEARNING OUTCOMES

By completing this assignment, the following concepts and practical skills were understood.

1. Understanding Software Testing Measurements

Learned how test execution, pass rate, defect density, defect closure, and requirement coverage are used to monitor software quality.

2. Preparing Status Reports

Understood how to prepare RAG summaries, metric-chart reports, and tool-generated dashboards for different audiences.

3. Monitoring and Controlling Testing

Learned how to calculate schedule gaps, identify delays, and compare corrective actions.

4. Defining Testing Completion Criteria

Understood why test execution, defect trends, and risk-based conditions must be considered before stopping testing.

5. Software Configuration Management

Learned how Git, release tags, baselines, and change-control procedures help maintain correct software versions.

6. Review Techniques

Understood the differences between peer desk-checks, technical reviews, and formal inspections.

7. Review Planning

Learned how to select work products for review and calculate the effort required within a project limit.

8. Review Reporting

Understood how to record comments, assign owners, track actions, and verify corrections.

9. Integrated Quality Control

Learned how measurements, reports, configuration audits, reviews, and stopping criteria work together.

10. Quality Control Adaptation

Understood how testing processes must change when the release cycle, team structure, technology, and regulatory requirements change.

RECOMMENDATIONS FOR IMPROVING THE SURECLAIM PLATFORM

Based on the analysis performed in this assignment, the following recommendations can improve the quality and reliability of SureClaim.

1. Use a Consistent Measurement System

The project should maintain a standard set of testing metrics for every release. Test execution, pass rate, open critical defects, defect closure rate, requirement coverage, and defect density should be monitored regularly.

2. Improve Early Warning

Schedule and quality problems should be identified as early as possible. Dashboards and automatic reports can help the release manager identify delays before they affect the final release date.

3. Strengthen Defect Management

Every defect should be recorded with its severity, priority, owner, and status. Critical defects should receive immediate attention and should not be ignored to meet schedule targets.

4. Improve Configuration Control

All important configuration items should be maintained in a controlled repository. Production deployments should be generated from approved release baselines.

5. Introduce Risk-Based Reviews

High-risk work products, especially fraud rules, claim settlement logic, payment handling, and security-related functions, should receive formal reviews.

6. Improve Test Automation

Repeated test cases can be automated to reduce manual effort and provide faster feedback. Automated testing should be included in the development pipeline.

7. Maintain Complete Audit Evidence

Each release should have a record of the build, configuration, test results, defects, reviews, and approval decision. This will support regulatory requirements and future investigations.

8. Improve Vendor Coordination

All development vendors should follow common coding, testing, configuration, and review procedures. Shared tools should be used to maintain consistent project information.

9. Verify External Service Integration

The external AI fraud-scoring service and UPI payment integration should be tested under normal, failure, and delayed-response conditions.

10. Maintain Human Control Over Release Decisions

Automated tools and AI can support testing and reporting, but important release decisions should be approved by responsible human reviewers.

FUTURE ENHANCEMENTS

The SureClaim quality control process can be improved further through the following enhancements.

Automated Quality Gates: Automated quality gates can check build status, test results, code review approvals, and configuration versions before deployment.

Continuous Testing: Testing can be performed throughout development rather than waiting until the end of a release.

Improved Risk Prediction: Historical defect information can be used to identify modules that may require additional testing attention.

Automated Configuration Auditing: The deployed configuration can be compared automatically with the approved release baseline.

AI-Assisted Review Support: AI can help identify possible code issues, summarize review comments, and prepare draft reports. All generated results should be verified.

Better Test Data Management: Controlled test data can improve repeatability and reduce errors caused by incomplete or inconsistent information.

Improved Release Monitoring: Production monitoring can help identify unexpected behaviour after deployment and provide feedback for future releases.

FINAL CONCLUSION

This assignment provided a comprehensive understanding of software testing activities and their importance in maintaining the quality of a software release. By using the SureClaim Motor

Insurance Claims Platform as the main example, the assignment covered measurement frameworks, status reporting, test monitoring, control responses, stopping criteria, configuration management, review techniques, review planning, review reporting, integrated quality control, and process adaptation.

The calculations and comparisons demonstrated that software testing must be monitored using suitable measurements. Test execution percentage, pass rate, defect density, defect closure rate, and requirement coverage provide useful information about the progress and quality of an application. The weekly data also demonstrated that testing progress may improve while important quality risks remain unresolved.

The assignment highlighted the importance of communicating testing information through suitable reports and dashboards. It also showed that testing delays should be investigated carefully before additional resources are introduced or test scope is reduced. Corrective actions should focus on the actual causes of delays while protecting important business and security requirements.

Configuration management and review activities were also found to be essential. Maintaining approved versions, recording changes, reviewing critical work products, and tracking corrective actions help prevent errors from reaching production. Proper documentation and audit evidence also make the testing process more transparent and reliable.

The final task demonstrated that quality control practices must adapt to changes in technology and project requirements. The move towards fortnightly releases, continuous delivery, UPI payouts, external AI fraud scoring, and outsourced development requires automated quality gates, controlled configuration changes, frequent reviews, and release-level evidence.

Finally, the assignment improved the understanding that software testing is not limited to finding defects. It is a continuous management activity that helps monitor progress, control risks, maintain configuration consistency, verify requirements, and support release decisions. A well-planned quality control process helps organizations deliver software that is more reliable, secure, maintainable, and suitable for user requirements.